УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Структуры данных: последовательности — списки, кортежи, срезы, включения

ЛЕКЦИЯ 6

Ахметов Ильнур Вазирович

План лекции

1. Последовательности: list, tuple, str, range — общее и отличия.
2. Списки (list): создание, методы, изменяемость, ловушки.
3. Кортежи (tuple): неизменяемость, упаковка и распаковка.
4. Индексация и работа со срезами (slices).
5. Списковые включения (list comprehensions) и генераторы.
6. Кейсы из разных отраслей: финансы, ритейл, медицина, логистика, спорт…

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы знаем из лекции 5

- for и while — программа научилась повторять действия.
- range() — числовые ряды, enumerate() и zip() — индексы и пары.
- Итераторы — ленивые вычисления, файл — тоже итератор.
- List comprehension — короткая запись создания списка.
- Но мы работали с данными «на лету», не сохраняя их.
- Сегодня: как СОЗДАВАТЬ и ХРАНИТЬ коллекции — последовательности.

консоль Python

```
>>> for w in wells:      # a wells откуда?
...     print(w)

>>> wells = ['К-12', 'К-13', 'К-14']
>>> fund = (85.4, 0.0, 12.7)
>>> code = 'К-12-Д'
>>> nums = range(1, 6)
```

Что такое последовательность

- Последовательность (sequence) — упорядоченный набор элементов.
- Упорядоченность: у каждого элемента есть НОМЕР (индекс).
- Встроенные последовательности Python:
- list — список: изменяемый, элементы любых типов.
- tuple — кортеж: неизменяемый.
- str — строка: неизменяемая последовательность символов.
- range — арифметическая последовательность чисел.

код

```
wells = ['К-12', 'К-13', 'К-14']   # list
point = (25.0, 18.5)               # tuple
code  = 'К-12-Д'                   # str
nums  = range(1, 6)                # range

# общее: индексы 0,1,2... и поддержка срезов
```

Общие операции над последовательностями

- len(seq) — количество элементов.
- seq[i] — обращение по индексу.
- seq[a:b] — срез (подпоследовательность).
- x in seq — проверка вхождения.
- for x in seq — итерация.
- seq + seq2 — конкатенация; seq * n — повторение.
- Одни и те же операции для списков, кортежей, строк.

консоль Python

```
>>> orders = [101, 102, 103]
>>> len(orders)
3
>>> orders[1]
102
>>> orders[1:3]
[102, 103]
>>> 102 in orders
True
>>> [0] * 3
[0, 0, 0]
```

Список (list): создание

- Список — изменяемая упорядоченная коллекция.
- Литерал: [x1, x2, ...] — квадратные скобки.
- Элементы любых типов, можно вперемешку.
- Пустой список: [] или list().
- Из другого объекта: list('абв') → ['а','б','в'].
- Генератор: [x**2 for x in range(5)].

консоль Python

```
>>> cities = ['Уфа', 'Казань', 'Тюмень']
>>> nums = [1, 4, 9, 16]
>>> mixed = ['К-12', 85.4, True]
>>> empty = []
>>> list('К-12')
['К', '-', '1', '2']
>>> list(range(3))
[0, 1, 2]
```

Изменяемость: главное отличие списка

- Список МОЖНО изменить после создания — изменяемый (mutable) тип.
- Замена элемента: lst[0] = 'новое'.
- Можно добавлять, удалять, переставлять элементы.
- Кортеж и строка — нельзя (см. далее).
- Изменяемость = гибкость, но и источник ошибок.

консоль Python

```
>>> sku = ['A', 'B', 'C']
>>> sku[0] = 'X'
>>> sku
['X', 'B', 'C']

>>> # а со строкой так нельзя:
>>> s = 'K-12'
>>> s[0] = 'M'
TypeError: 'str' object does not
support item assignment
```

Методы добавления: append, insert, extend

append(x) один элемент в конец.

insert(i, x) вставить по индексу i (сдвиг вправо).

extend(seq) все элементы другой последовательности.

- append(['a','b']) добавит ОДИН элемент-список, extend — два!
- Все три меняют список НА МЕСТЕ и возвращают None.

консоль Python

```
>>> log = ['start']
>>> log.append('query')
>>> log.insert(0, 'boot')
>>> log
['boot', 'start', 'query']
>>> log.extend(['render', 'stop'])
>>> log
['boot','start','query','render','stop']
```

Методы удаления: pop, remove, clear

pop() удалить и ВЕРНУТЬ последний элемент.

pop(i) удалить и вернуть элемент по индексу i.

remove(x) удалить ПЕРВОЕ вхождение по значению.

clear() очистить список полностью.

- pop — по индексу и возвращает; remove — по значению и ничего не возвращает.
- remove несуществующего значения → ValueError.

консоль Python

```
>>> queue = ['a', 'b', 'c']
>>> last = queue.pop()
>>> last
'c'
>>> queue.remove('a')
>>> queue
['b']
>>> queue.clear()
>>> queue
[]
```

Методы поиска и сортировки

index(x) индекс первого вхождения (ValueError, если нет).

count(x) сколько раз встречается x.

sort() сортирует НА МЕСТЕ (возвращает None!).

reverse() переворачивает на месте.

- sorted(seq) — НОВЫЙ отсортированный список, оригинал не меняет.
- sort(key=..., reverse=True) — по ключу, по убыванию.

консоль Python

```
>>> rates = [85.4, 0.0, 12.7]
>>> rates.index(12.7)
2
>>> rates.count(0.0)
1
>>> rates.sort()
>>> rates
[0.0, 12.7, 85.4]
>>> sorted(rates, reverse=True)
[85.4, 12.7, 0.0]
```

Ловушка: sort() возвращает None

- Самая частая ошибка новичков со списками.
- d = d.sort() → d станет None!
- Методы, меняющие на месте, возвращают None.
- append, insert, extend, remove, sort, reverse, clear.
- sort() — без присваивания; sorted() — с присваиванием.

консоль Python

```
>>> d = [85.4, 0.0, 12.7]
>>> d = d.sort()      # ОШИБКА!
>>> d
None
>>> d.sort()          # верно: без присваивания
>>> d
[0.0, 12.7, 85.4]
>>> s = sorted(d)     # верно: sorted
>>> s
[0.0, 12.7, 85.4]
```

Копирование списка: ловушка ссылок

- a = b НЕ копирует список — это ДВЕ ССЫЛКИ на один объект!
- Изменение через a изменит и b — одна память.
- Копия: lst.copy(), lst[:], list(lst).
- Вложенные структуры — глубокая копия copy.deepcopy().
- Классическая ловушка на собеседованиях и в коде.

консоль Python

```
>>> a = [1, 2, 3]
>>> b = a           # ССЫЛКА, не копия
>>> b.append(4)
>>> a
[1, 2, 3, 4]        # a тоже изменился!

>>> c = a.copy()    # настоящая копия
>>> c.append(5)
>>> a
[1, 2, 3, 4]        # a не изменился
```

Практика: реестр операциями списка

- Задача: собрать реестр по мере поступления данных.
- Начинаем с пустого списка, наполняем append.
- Сортированная копия — sorted() (оригинал не трогаем).
- Ключевой навык — вести ЖУРНАЛ операций через список.

код

```
registry = []
for code in ["К-12", "К-13", "К-14"]:
    registry.append(code)

print(f"Всего: {len(registry)}")
print(registry)      # ['К-12','К-13','К-14']

ordered = sorted(registry)
print(f"Первая: {ordered[0]}")
```

Практика: анализ дебитов методами списка

- Задача: по списку дебитов — сумма, средний, максимум, минимум.
- Встроенные: sum(), len(), max(), min().
- Ноль = остановленная скважина — исключаем из среднего.
- Средний дебит работающих — ключевой показатель фонда.

код

```
debits = [85.4, 0.0, 12.7, 65.3, 0.0, 45.1]

active = [d for d in debits if d > 0]

print(f"Всего:      {len(debits)}")
print(f"Работающих: {len(active)}")
print(f"Сумма: {sum(debits):.1f} т/сут")
print(f"Средн. по раб.:
{sum(active)/len(active):.2f}")
print(f"Максимум: {max(debits):.1f}")
print(f"Минимум по раб.: {min(active):.1f}")
```

Вложенные списки: матрица

- Список может содержать другие списки — вложенные структуры.
- Матрица = список строк, каждая строка — список чисел.
- Доступ: matrix[i][j] — строка i, столбец j.
- Так хранят таблицы; numpy/pandas — поверх той же идеи.

код

```
# 3 скважины × 3 суток
matrix = [
    [85.4, 84.1, 83.0],
    [0.0,  0.0,  0.0],
    [12.7, 12.5, 12.1],
]
print(matrix[0][1])      # 84.1

for row in matrix:
    print(f"Сумма: {sum(row):.1f}")
```

Кортеж (tuple): неизменяемая последовательность

- Кортеж — упорядоченная НЕИЗМЕНЯЕМАЯ коллекция.
- Литерал: (a, b, c) — круглые скобки.
- Из одного элемента: (x,) — ОБЯЗАТЕЛЬНО запятая!
- Без запятой (x) — просто x в скобках, не кортеж.
- Пустой кортеж: () или tuple().

консоль Python

```
>>> point = (55.7, 54.9)
>>> point
(55.7, 54.9)
>>> single = (42,)
>>> type(single)
<class 'tuple'>
>>> not_tuple = (42)
>>> type(not_tuple)
<class 'int'>
```

Неизменяемость кортежа

- Кортеж нельзя изменить после создания.
- Присвоить элемент → TypeError.
- append, remove, sort — методов просто нет.
- Зато можно читать: индексы, срезы, len, in, итерация.
- Зачем: безопасность + возможность быть ключом словаря.

консоль Python

```
>>> point = (25.0, 18.5)
>>> point[0]
25.0
>>> point[0] = 30.0
TypeError: 'tuple' object does not
support item assignment

>>> point.append(1)
AttributeError: 'tuple' object has no
attribute 'append'
```

Упаковка (packing)

- Упаковка — объединение значений в кортеж.
- Перечислите через запятую: a = 1, 2, 3 — это кортеж!
- Скобки не обязательны (но желательны для читаемости).
- return x, y из функции — тоже упаковка.
- Так функция возвращает несколько значений сразу.

код

```
point = 55.7, 54.9
print(point)          # (55.7, 54.9)
print(type(point))    # <class 'tuple'>

def location():
    return 55.7, 54.9   # упаковка

result = location()
print(result)         # (55.7, 54.9)
```

Распаковка (unpacking)

- Распаковка — присвоение элементов переменным.
- Число переменных должно совпадать с длиной кортежа.
- Иначе — ValueError: too many / not enough values.
- Звёздочка (*rest) собирает остаток в список.
- Работает со списками и строками тоже.

консоль Python

```
>>> point = (55.7, 54.9)
>>> lat, lon = point
>>> lat
55.7
>>> lon
54.9

>>> a, *rest = (1, 2, 3, 4)
>>> rest
[2, 3, 4]           # остаток — список
```

Обмен значений через кортеж

- Классика: обмен двух переменных без временной.
- a, b = b, a — благодаря упаковке и распаковке.
- В других языках нужна третья переменная temp.
- Тот же приём — перестановка элементов списка.

консоль Python

```
>>> a, b = 1, 2
>>> a, b = b, a
>>> a
2
>>> b
1

>>> lst = [1, 2, 3]
>>> lst[0], lst[2] = lst[2], lst[0]
>>> lst
[3, 2, 1]
```

Когда кортеж, а когда список

Кортеж данные фиксированы: координаты, запись из БД, пары.

Список коллекция меняется: реестр, журнал, фильтр, сортировка.

Кортеж нужен ключ словаря (список ключом быть не может).

Кортеж возврат нескольких значений из функции.

- Правило: если набор не меняется — кортеж; иначе список.

код

```
# кортеж: запись фиксирована
well = ("К-12", 85.4, 0.24)

# список: фонд меняется
fund = ["К-12", "К-13", "К-14"]
fund.append("К-15")   # ОК

# кортеж как ключ словаря
coords = {(55.7, 54.9): 'Куст 3'}   # ОК
```

Практика: кортеж как запись скважины

- Храним запись скважины как неизменяемый кортеж.
- Состав: ID, дебит, обводнённость.
- Распаковываем в переменные для удобства.
- Список таких кортежей — компактный реестр.
- Кортеж легче списка по памяти.

код

```
fund = [
    ("К-12", 85.4, 0.24),
    ("К-13",  0.0, 0.00),
    ("К-14", 12.7, 0.91),
]

for wid, debit, water in fund:
    st = "активна" if debit > 0 else "стоп"
    print(f"{wid}: {debit:5.1f} т/сут,
W={water:.0%}, {st}")
```

Индексация: доступ по номеру

- Индексация начинается с НУЛЯ.
- seq[0] — первый, seq[len-1] — последний.
- Отрицательные: seq[-1] — последний, seq[-2] — предпоследний.
- Выход за границы → IndexError.
- Относится к спискам, кортежам, строкам, range.

консоль Python

```
>>> w = ['K-12', 'K-13', 'K-14']
>>> w[0]
'K-12'
>>> w[-1]
'K-14'        # последний
>>> w[-2]
'K-13'
>>> w[3]
IndexError: list index out of range
```

Изменение по индексу (только списки)

- lst[i] = x — заменить элемент по индексу.
- Только для изменяемых (list). Кортеж/строка → TypeError.
- Несколько сразу через срез: lst[1:3] = [...].
- Индекс должен существовать — иначе IndexError.

консоль Python

```
>>> d = [85.4, 0.0, 12.7]
>>> d[0] = 90.0
>>> d
[90.0, 0.0, 12.7]

>>> d[1:3] = [10.0, 20.0]
>>> d
[90.0, 10.0, 20.0]

>>> s = 'abc'; s[0] = 'x'
TypeError
```

Срез: seq[start:stop:step]

- seq[start:stop:step] — подпоследовательность.
- start — с какого индекса (включая).
- stop — до какого (НЕ включая!) — как range.
- step — шаг (по умолчанию 1).
- Все части необязательны: seq[:] — копия.
- Срез всегда возвращает НОВЫЙ объект.

консоль Python

```
>>> w = ['K-12','K-13','K-14','K-15']
>>> w[1:3]
['K-13', 'K-14']      # индексы 1, 2
>>> w[:2]
['K-12', 'K-13']      # с начала
>>> w[2:]
['K-14', 'K-15']      # до конца
>>> w[::2]
['K-12', 'K-14']      # каждый второй
```

Отрицательные индексы в срезах

- В срезах можно использовать отрицательные индексы.
- w[-2:] — два последних элемента.
- w[:-2] — всё, кроме двух последних.
- w[::-1] — РАЗВОРОТ последовательности (шаг -1).
- Частый приём — развернуть список одной записью.

```
консоль Python
>>> w = ['К-12','К-13','К-14','К-15']
>>> w[-2:]
['К-14', 'К-15']      # два последних
>>> w[:-2]
['К-12', 'К-13']
>>> w[::-1]
['К-15','К-14','К-13','К-12']
>>> 'К-12-Д'[::-1]
'Д-21-К'
```

Шаг среза: выборка с интервалом

- step > 0 — вперёд, step < 0 — назад.
- seq[::2] — каждый второй (0, 2, 4...).
- seq[1::2] — каждый второй с 1 (1, 3, 5...).
- seq[::-2] — каждый второй с конца.
- Практика: прореживание ряда измерений.

консоль Python

```
>>> n = [0,1,2,3,4,5,6,7,8,9]
>>> n[::2]
[0, 2, 4, 6, 8]
>>> n[1::2]
[1, 3, 5, 7, 9]
>>> n[::-2]
[9, 7, 5, 3, 1]
>>> n[2:8:3]
[2, 5]
```

Срезы строк и кортежей

- Срезы работают одинаково для всех последовательностей.
- Строка: код 'К-12-Д' → части через срезы.
- Кортеж: подкортеж через срез.
- Срез сохраняет ТИП: срез строки — строка, кортежа — кортеж.

консоль Python

```
>>> code = 'К-12-Д'
>>> code[:4]
'К-12'
>>> code[2:4]
'12'
>>> code[-1]
'Д'

>>> t = (10, 20, 30, 40)
>>> t[1:3]
(20, 30)            # кортеж
```

Присваивание через срез (только списки)

- lst[a:b] = [...] — заменить часть списка.
- Количество может НЕ совпадать — длина изменится!
- lst[1:2] = [1,2,3] — вставит три вместо одного.
- lst[1:3] = [] — удалит элементы.
- То же делает del lst[a:b].

консоль Python

```
>>> d = [10, 20, 30, 40]
>>> d[1:3] = [99]
>>> d
[10, 99, 40]

>>> d = [10, 20, 30, 40]
>>> d[1:2] = [1, 2, 3]
>>> d
[10, 1, 2, 3, 30, 40]

>>> del d[1:3]
>>> d
[10, 3, 30, 40]
```

Практика: скользящее окно

- Сгладить ряд скользящим средним (окно 3).
- Срез window[i:i+3] вырезает окно.
- Количество окон: len - width + 1.
- Основа сглаживания и поиска трендов.

код

```
data = [100, 102, 98, 105, 110, 108]
width = 3
smoothed = []

for i in range(len(data) - width + 1):
    w = data[i:i + width]
    smoothed.append(sum(w) / width)

print([f"{v:.1f}" for v in smoothed])
# ['100.0','101.7','104.3','107.7']
```

Практика: последние записи и безопасность

- Частая задача: взять последние N записей ряда.
- data[-7:] — последние семь; data[:7] — первые семь.
- Срез безопасен: если элементов меньше — вернёт сколько есть (БЕЗ ошибки!).
- Отличие от индекса: w[100] упадёт, а w[:100] — нет.

код

```
hourly = [120,118,121,119,122,120,117,115]

print(f"Первые сутки: {hourly[:7]}")
print(f"Последние:    {hourly[-3:]}")

print(hourly[:100])   # весь список, без ошибки

try:
    print(hourly[100])
except IndexError:
    print("Индекс 100 вне диапазона")
```

Списковое включение: синтаксис

- [выражение for элемент in последовательность]
- Создаёт НОВЫЙ список, применяя выражение к каждому.
- Короче и быстрее цикла с append.
- Можно условие, несколько for.
- Читается: «список таких-то для каждого из таких-то».

код

```
squares = []
for x in range(1, 6):
    squares.append(x ** 2)

squares = [x ** 2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

# та же логика, но одна строка
```

Условие во включении (фильтрация)

- [выражение for x in seq if условие] — фильтр.
- Элемент попадает, только если условие истинно.
- Условие можно комбинировать: if a and b.
- Два условия — два if подряд.
- Аналог filter(), но читаемее.

код

```
debits = [85.4, 0.0, 12.7, 65.3, 0.0, 45.1]

active = [d for d in debits if d > 0]
# [85.4, 12.7, 65.3, 45.1]

high = [d for d in debits if d > 50]
# [85.4, 65.3]

mid = [d for d in debits if 10 < d < 70]
# [12.7, 65.3, 45.1]
```

if-else внутри выражения (преобразование)

- if ПОСЛЕ for — фильтрует; if-else ДО for — преобразует.
- [x if условие else y for x in seq] — тернарник.
- Фильтр отбрасывает, тернарник заменяет значение.
- Каждый элемент остаётся, меняется его значение.
- Запомните позицию — это частая путаница!

код

```
debits = [85.4, 0.0, 12.7]

# ФИЛЬТР: нули отброшены (2 элемента)
a = [d for d in debits if d > 0]
# [85.4, 12.7]

# ТЕРНАРНИК: все 3, нули -> None
b = [d if d > 0 else None for d in debits]
# [85.4, None, 12.7]

c = ["работает" if d > 0 else "стоп" for d in
debits]
```

Вложенные включения (две for)

- [f(x,y) for x in A for y in B] — произведение.
- Порядок: внешний for — первый, внутренний — второй.
- Результат: len(A) × len(B) элементов.
- Плоский список: [x for row in m for x in row].
- Читайте слева направо — первый for внешний.

код

```
wells = ["K-12", "K-13"]
days = [1, 2]
pairs = [(w, d) for w in wells for d in days]
# [('K-12',1),('K-12',2),('K-13',1),('K-13',2)]

matrix = [[1, 2], [3, 4]]
flat = [x for row in matrix for x in row]
# [1, 2, 3, 4]
```

Практика: включения в работе с фондом

- Из реестра скважин — нужные выборки.
- ID активных, округлённые дебиты, словарь.
- Одна строка вместо цикла + append.
- Включения бывают и для словарей {k: v}.
- Осторожно: создаёт список в памяти.

код

```
wells = [
    {"id": "К-12", "дебит": 85.4, "W": 0.24},
    {"id": "К-13", "дебит": 0.0,  "W": 0.00},
    {"id": "К-14", "дебит": 120.5,"W": 0.88},
    ]
active_ids = [w["id"] for w in wells if
w["дебит"] > 0]
# ['К-12', 'К-14']

rounded = [round(w["дебит"]) for w in wells]
# [85, 0, 120]
```

Генераторные выражения: лениво

- (выражение for x in seq) — КРУГЛЫЕ скобки.
- Не создаёт список — возвращает генератор.
- Память O(1) вместо O(n).
- Одноразовый: второй обход даст пустоту.
- Идеален для sum, max, min, any, all.
- Внутри вызова скобки не обязательны.

код

```
sq = [x**2 for x in range(1_000_000)]      # 8 МБ
gen = (x**2 for x in range(1_000_000))     # 48
байт

total = sum(x**2 for x in range(1_000_000))

print(max(d for d in debits if d > 0))
print(any(w["W"] > 0.8 for w in wells))
```

Словарные и множественные включения

- Включения не только для списков.
- Словарь: {k: v for x in seq} — ключ-значение.
- Множество: {x for x in seq} — уникальные значения.
- Синтаксис как у списка, но скобки задают тип.
- Множество убирает дубликаты автоматически.

код

```
wells = [("K-12", 85.4), ("K-13", 0.0),
         ("K-14", 12.7)]

# словарь:
debit_map = {wid: d for wid, d in wells}
# {'K-12': 85.4, 'K-13': 0.0, 'K-14': 12.7}

# множество (уникальные):
nums = [5, 1, 5, 3, 1, 3, 9, 7]
unique = {n for n in nums}
# {1, 3, 5, 7, 9}
```

Финансы: портфель акций и доходность

- Цены закрытия акции за неделю (список чисел).
- Дневная доходность — через включение по соседним.
- Срезы: первая и последняя цена недели.
- Включения считают весь ряд доходностей разом.

код

```
prices = [101.5, 103.2, 102.1, 105.7, 104.3]

returns = [
    round((prices[i] - prices[i-1]) / prices[i-1]
* 100, 2)
    for i in range(1, len(prices))
]

print(f"Доходности: {returns} %")
print(f"Лучший день: +{max(returns)}%")
print(f"Неделя: {prices[0]} -> {prices[-1]}")
```

Ритейл: корзина покупок (кортежи)

- Корзина — список кортежей (товар, цена, количество).
- Сумма — через генератор: sum(цена*кол-во).
- Распаковка кортежей прямо в цикле/генераторе.
- max по ключу — самый дорогой товар.

код

```
cart = [
    ("Молоко", 89.9, 2),
    ("Хлеб",   45.0, 1),
    ("Сыр",   540.0, 1),
]

total = sum(price * qty for _, price, qty in
cart)
top = max(cart, key=lambda item: item[1])

print(f"Итого: {total:.2f} руб")
print(f"Самый дорогой: {top[0]}")
```

Медицина: температура пациента

- Ряд измерений температуры каждые 3 часа.
- Срезы: ночные и дневные замеры.
- Включение: найти повышения (>37.5).
- Статистика: max по дневным.

код

```
temp = [36.6, 36.8, 37.9, 38.5, 38.1, 37.2, 36.7,
36.5]

night = temp[:2] + temp[6:]   # ночные
day = temp[2:6]               # дневные

fever = [t for t in temp if t > 37.5]

print(f"Дневной максимум: {max(day)}")
print(f"Повышений: {len(fever)}")
```

Логистика: маршрут из координат

- Маршрут — список кортежей (широта, долгота).
- Распаковка кортежей в переменные lat, lon.
- Срез — участок маршрута.
- Кортеж защищает координаты от случайной порчи.

код

```
route = [
    (55.7558, 37.6173),  # старт
    (55.7887, 37.6380),
    (55.7512, 37.6013),  # финиш
]

for lat, lon in route:
    print(f"({lat:.4f}, {lon:.4f})")

seg = route[:2]   # участок маршрута
print(f"Точек: {len(route)}, старт: {route[0]}")
```

Погода: температуры месяца

- Дневные температуры июня (30 значений).
- Срезы — декады (по 10 дней).
- Включение — жаркие дни.
- sum/len — средняя температура.

код

```
june = [18,21,24,26,29,31,33,32,28,24,
        20,19,22,25,27,30,32,31,27,25,
        21,20,23,26,28,29,30,28,24,22]

hot = [t for t in june if t > 28]
dec = [june[:10], june[10:20], june[20:]]

print(f"Средняя: {sum(june)/len(june):.1f}°C")
print(f"Жарких дней: {len(hot)}")
print(f"I декада: {sum(dec[0])/10:.1f}°C")
```

Спорт: турнирная таблица

- Список кортежей (команда, очки).
- Сортировка копии по очкам — sorted с key.
- Включение — лидеры (очки >= 25).
- enumerate — места с первого.

код

```
table = [("Спартак", 24), ("Зенит", 30),
         ("ЦСКА", 22), ("Локомотив", 27)]

ranked = sorted(table, key=lambda x: x[1],
                reverse=True)

leaders = [t for t, p in ranked if p >= 25]

for i, (team, pts) in enumerate(ranked, 1):
    print(f'{i}. {team}: {pts}')
```

Образование: оценки группы

- Оценки группы — вложенный список (студент × предмет).
- Включение — средний балл по каждому студенту.
- Внутренний генератор — число «хороших» оценок.
- Вложенные списки = таблица = строки и столбцы.

код

```
grades = [
    [5, 4, 5, 3],   # Иванов
    [4, 4, 4, 4],   # Петров
    [5, 5, 5, 5],   # Сидоров
]

avg = [round(sum(g) / len(g), 2) for g in grades]
good = [sum(1 for m in g if m >= 4) for g in
grades]

for i, a in enumerate(avg, 1):
    print(f'Студент {i}: средний {a}')
```

Веб-аналитика: лог посещений

- Список статусов ответов сервера (200, 404, 500...).
- Генератор + sum — счётчик ошибок без промежуточного списка.
- Срезы — первые записи лога.
- Полезно для мониторинга доступности сервиса.

код

```
statuses =
[200,200,404,200,500,200,404,200,503,200]

errors = sum(1 for s in statuses if s >= 400)
ok = sum(1 for s in statuses if s == 200)

print(f"Запросов: {len(statuses)}")
print(f"Успешных: {ok}, Ошибок: {errors}")
print(f"Первые 5: {statuses[:5]}")
```

ЗАКРЕПИМ

Типичные ошибки и ловушки

d = d.sort()	sort() возвращает None — d станет None. Используйте sorted().
a = b для списка	это ссылка, не копия. Копия: b = a.copy() или a[:].
Удаление в цикле	for x in lst: lst.remove(x) — пропуски. Создавайте новый список.
(x) вместо (x,)	кортеж из одного элемента требует запятой.
Срез vs индекс	w[:100] не упадёт, w[100] → IndexError.
if после for	фильтрует элементы; тернарник ДО for — преобразует значения.

Итоги лекции

1. list — изменяемая последовательность; tuple — неизменяемая.
2. Методы списка меняют НА МЕСТЕ и возвращают None (sort, append).
3. a = b для списка — ссылка; копия нужна явно (.copy(), [:]).
4. Упаковка и распаковка кортежей: a, b = b, a — обмен без temp.
5. Индексы с 0, отрицательные — с конца; срез [start:stop:step] — новый объект.
6. Включения: фильтр if после for, тернарник до for; генератор — ленивый.
7. Одни и те же приёмы работают в нефтянке, финансах, ритейле, медицине…

Вопрос 1 из 6

Чем список отличается от кортежа? Приведите пример, где нужен именно кортеж.

Вопрос 2 из 6

Что выведет: a = [1,2,3]; b = a; b.append(4); print(a)? Почему?

Вопрос 3 из 6

В чём разница между d.sort() и sorted(d)? Что вернёт первая?

Вопрос 4 из 6

Что выведет [1,2,3,4,5,6][1::2] и [1,2,3,4,5,6][::-1]?

Вопрос 5 из 6

Чем отличается [x for x in s if x > 0] от [x if x > 0 else 0 for x in s]?

Вопрос 6 из 6

Почему (x for x in range(5)) не занимает память, а [x for x in range(5)] — да?